RBAC and Logging in OpenHospital

Giuseppe Pisano
g.pisano@unibo.it

Healthcare Data Privacy and Security

August, 2026

S.K.I.L.L.E.D — AID013244/04/2
Strategic Knowledge and Inclusive Lifelong Learning in the health sector for youth Employability and Development

Roadmap I

Roadmap II

Lab Goal

What we will do

We will install a test OpenHospital instance, enable multi-user login, configure groups and users, test which functions each group can access, and inspect what the application logs actually record.

Why OpenHospital

OpenHospital is a real healthcare application. Its access-control model is **RBAC-like**: users belong to groups, and groups receive menu/function permissions from an administrator.

How This Lab Progresses

Learning path

We move from **installation**, to **first run**, to **multi-user mode**, then to **group policy**, **user accounts**, **access testing**, and finally to **logging and audit-gap discussion**.

Hospital storyline

St. Isidore Hospital is piloting OpenHospital. The IT team must create distinct accounts for registration staff, cashiers, clinicians, laboratory staff, pharmacy staff, and auditors.

Lab Safety

Important

Use only a local test installation with demo or artificial data. Do not use real patient data, personal staff credentials, or production systems during this lab.

Operational rule

If something goes wrong, delete the test folder or database and start again. The purpose is to learn policy configuration, not to preserve a production deployment.

Preparation

Section Context: Preparation

What we are talking about

Before touching access control, we need a working OpenHospital environment. The simplest option for this lab is the portable/test setup because it avoids a full client/server deployment.

What OpenHospital Is

System profile

OpenHospital is a free and open-source hospital information system developed in Java. It supports standalone portable use and client/server configurations with a central database [Informatici Senza Frontiere Onlus, 2026a].

Healthcare relevance

Its modules include patient registration, pharmacy, laboratory, outpatient management, admissions, billing, and statistics. That gives us realistic functions for access-control testing.

Recommended Lab Setup

Use portable mode first

Use **portable mode** or a single-machine test setup. It is easier to reset and it keeps the focus on users, groups, permissions, and logs.

Why not start with client/server

Client/server mode is valuable later, but it adds database networking, host firewalls, and client configuration before the access-control workflow is clear.

Prerequisites

Before starting

- Download the latest OpenHospital release for your operating system.
- Unpack the archive into a writable folder.
- Check that the package includes its bundled JRE and portable MariaDB/MySQL server.
- Keep one untouched copy so the lab can be reset quickly.

Source

The OpenHospital administrator guide describes download, unpacking, installation modes, and Java requirements [Informatici Senza Frontiere Onlus, 2026a].

Java Runtime

Default case

The portable OpenHospital package normally includes its own Java Runtime Environment, so Java should not need a separate installation for the lab.

If Java is missing

If the startup script reports that Java cannot be found, install a supported Java runtime for the OpenHospital release, or configure the script variable `JAVA_BIN` to point to a system Java executable.

Folder Discipline

Practical habit

Create a dedicated lab folder such as `OpenHospital-Lab`. Avoid spaces and unusual characters in the path if possible. Make sure the user running OpenHospital has read/write permission on the extracted folder.

Hospital example

St. Isidore IT keeps the pilot under a clearly named test directory, separate from real hospital systems and backups.

Checkpoint: Preparation

You are ready when

- OpenHospital is downloaded.
- The package is fully unpacked.
- The folder is writable.
- You know how to start the application on your operating system.

First Run

Section Context: First Run

What we are talking about

The first run confirms that the application works before we modify security settings. This creates a known-good baseline.

Practical rule

Do not begin by editing configuration files. First prove that the application starts in its default state.

Start OpenHospital

Action

Run the provided OpenHospital startup script for your operating system from the extracted folder. Use the official startup script rather than double-clicking random internal files.

Expected observation

On Linux, run `./oh.sh`. On Windows, run `oh.bat`; it launches `oh.ps1` and shows the interactive menu. OpenHospital should then show its startup window or login window.

Startup Menu

Useful options

The portable startup menu can select **PORTABLE**, **CLIENT**, or **SERVER** mode, change language, toggle debug logging, initialize demo data, test database settings, and reset the installation.

Lab choice

Use **PORTABLE** mode for this lab. Demo data can be useful for exploration, but enabling it may require deleting/resetting portable data.

Windows Startup Notes

If Windows blocks the script

The Windows launcher `oh.bat` starts `oh.ps1`. If PowerShell blocks it, open the file properties for `oh.ps1`, choose **Unblock**, then run it again.

Command-line fallback

From PowerShell, the README shows this form: `powershell.exe -ExecutionPolicy Bypass -File ./oh.ps1`.

Debug and Reset Options

Useful during the lab

The startup menu can toggle **DEBUG** logging, generate configuration files, initialize demo data, export or restore the database, and clean/reset the portable installation.

Warning

Use reset and demo initialization carefully: they may delete or overwrite portable data. This is acceptable only when working on a disposable lab copy.

Checkpoint: First Run

Record your baseline

- Did the application start?
- Did it ask for a login?
- Which menu areas are visible?
- Where is the configuration folder?
- Where are logs written?

Confirm Login Mode

Section Context: Login Mode

What we are talking about

To study RBAC-like behavior, OpenHospital must ask users to log in. In the portable package this may already be configured, so first observe the actual startup behavior.

Hospital lens

St. Isidore cannot manage accountability if all staff effectively act as the same user. Multi-user mode is the doorway to meaningful authorization and audit.

If Login Already Appears

Action

If OpenHospital starts with a login window, continue directly with the administrator account: `admin` / `admin`.

Security discussion

This is the expected behavior in many current portable packages. In a real deployment, default credentials must be changed immediately.

If Login Does Not Appear

Action

Use the startup script configuration. In the portable scripts, single/multi-user behavior is controlled by `OH_SINGLE_USER` on Linux or `$script:OH_SINGLE_USER` on Windows.

Expected setting

For this lab, the value should be `no`. The README also shows that configuration files can be generated from the startup scripts when needed.

Configuration Files

Where to look

The portable scripts create and manage runtime folders such as `data/conf`, `data/db`, `data/log`, `data/photo`, and OpenHospital resource files under `oh/rsc`.

Practical warning

Do not edit random files while OpenHospital is running. If you need to change startup behavior, stop the app, adjust the startup script or generated configuration intentionally, then restart.

Default Portable Folders

Folder	Purpose
`oh`	OpenHospital application files
`sql`	Database creation scripts
`data/conf`	MariaDB/MySQL configuration
`data/db`	Portable database files
`data/log`	Startup and application logs
`data/photo`	Patient photo storage
`data/dump`	Database exports/backups

Checkpoint: Login Mode

You have succeeded when

- OpenHospital shows a login prompt.
- The `admin` account can log in.
- The user/group management menu is reachable from Settings.
- You can explain why shared/default accounts are unsafe in production.

Design the RBAC Policy

Section Context: Policy Design

What we are talking about

Before clicking permissions, we design the policy. The administrator should know which job functions exist and which system functions each job needs.

Hospital lens

This is where system administration meets governance. The tool enforces settings, but the hospital decides the policy.

OpenHospital Model

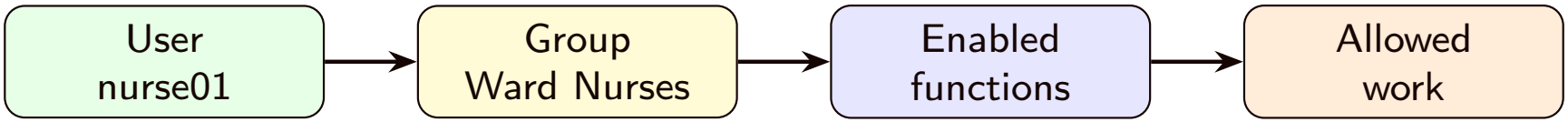
RBAC-like structure

OpenHospital manages users organized into groups. Each group is characterized by administrator assigned permissions [Informatici Senza Frontiere Onlus, 2026b].

Classification

Treat OpenHospital as **application-level RBAC-like access control**: users map to groups, and groups map to menu/function permissions.

Scheme: OpenHospital Access Model



Reading the scheme

Access is not delegated by file owners. It is centrally configured through group permissions.

Policy Scenario

Groups to create

- **Registration**: register and search patients.
- **Cashiers**: manage patient bills and payments.
- **Laboratory**: manage lab exams and results.
- **Pharmacy**: manage medicines and stock.
- **Auditors**: review data but avoid operational changes where possible.

Policy Matrix

Group	Patients	Billing	Lab	Pharmacy
Registration	Create/search	No	No	No
Cashiers	Read only	Create/update	No	No
Laboratory	Read only	No	Create/update	No
Pharmacy	Read only	No	No	Create/update
Auditors	Review	Review	Review	Review

Practical note

Exact menu names may differ. Map the policy intent to the nearest OpenHospital menu/function items.

Least Privilege Rule

Policy rule

Start from what each group needs for routine work. Do not enable extra functions just because they are convenient during setup.

Hospital example

Cashiers need billing functions. They do not need laboratory-result entry or pharmacy stock management.

Checkpoint: Policy Design

Before configuration

- List each group.
- Write the work each group must perform.
- Write one action each group must **not** perform.
- Decide which tests will prove the policy works.

Configure Groups and Users

Section Context: Configuration

What we are talking about

Now we translate policy into OpenHospital settings: groups, group menu permissions, and user accounts.

Hospital lens

The administrator is not writing access-control code. They are configuring the model already provided by the application.

Open Users and Groups

Action

Log in as `admin`. Open the Settings area and select **Users & Groups**. The OpenHospital user guide describes this menu as the place where groups and users are managed [Informatici Senza Frontiere Onlus, 2026b].

Expected observation

You should see controls for managing groups and users. If the menu is missing, check that multi-user mode is enabled and that the administrator has the relevant settings permissions.

Create Groups

DOCTORS WITH AFRICA CUAMM

Action

Create the lab groups: `registration`, `cashiers`, `laboratory`, `pharmacy`, and `auditors`. Use clear descriptions.

Expected observation

The groups should appear in the Groups Browser. Deleted or disabled groups should not be used for new users.

Configure Group Menu

Action

For each group, open **Group Menu**. Enable only the menu branches and leaf functions needed for that group. The user guide describes branches as menus/windows and leaves as buttons/functions
[Informatici Senza Frontiere Onlus, 2026b].

Practical technique

Configure one group at a time, then immediately test it with a matching user. This is slower but much easier to debug.

Create Users

Action

Create one test user per group:

User	Group	Test purpose
`reg01`	registration	Patient registration
`cash01`	cashiers	Billing
`lab01`	laboratory	Lab workflow
`pharm01`	pharmacy	Pharmacy workflow
`audit01`	auditors	Review and logging

Password and Account State

Action

Set simple temporary lab passwords, then verify whether each account is active, locked, or deleted. OpenHospital distinguishes locked users from deleted users in the user management view [Informatici Senza Frontiere Onlus, 2026b].

Security discussion

In real deployment, use password rules, remove default credentials, disable unused accounts, and review accounts periodically.

Checkpoint: Configuration

DOCTORS WITH AFRICA CUAMM

You have succeeded when

- Each group exists.
- Each group has a deliberately configured menu.
- Each test user belongs to the intended group.
- No test user is unintentionally locked or deleted.

Test Access

Section Context: Access Testing

What we are talking about

Access control is not finished when the policy is configured. We must test whether the system actually permits intended work and denies unintended work.

Hospital lens

Testing protects both safety and privacy. A nurse blocked from routine documentation is a care problem; a cashier who can edit lab results is a security problem.

Test Method

For each user

- Log out from the previous account.
- Log in as the test user.
- Record which menus and buttons are visible.
- Attempt one allowed task.
- Attempt one denied task.

Test Case: Registration

DOCTORS WITH AFRICA CUAMM

Expected allow

`reg01` should be able to access patient registration/search functions.

Expected deny

`reg01` should not be able to manage pharmacy stock, enter laboratory results, or configure users and groups.

Test Case: Cashier

Expected allow

`cash01` should be able to work with patient billing and payments.

Expected deny

`cash01` should not be able to change laboratory results, edit pharmacy stock, or create administrator accounts.

Test Case: Laboratory

Expected allow

`lab01` should be able to access laboratory workflow functions.

Expected deny

`lab01` should not be able to manage payments or broad system settings.

Test Case: Pharmacy

DOCTORS WITH AFRICA CUAMM

Expected allow

`pharm01` should be able to access pharmacy and stock workflow functions.

Expected deny

`pharm01` should not be able to register users, modify billing records, or configure group menus.

Test Case: Auditor

Expected allow

`audit01` should be able to review the information needed for oversight, depending on the permissions available in the installed OpenHospital version.

Expected deny

`audit01` should not perform routine operational changes such as entering lab results, changing stock, or editing bills unless the lab policy explicitly permits it.

Evidence Table

User	Allowed test	Denied test	Result
`reg01`	Register patient	Edit pharmacy stock	Pass / fail
`cash01`	Create bill	Enter lab result	Pass / fail
`lab01`	Enter lab result	Manage users	Pass / fail
`pharm01`	Manage stock	Manage billing	Pass / fail
`audit01`	Review data	Modify operations	Pass / fail

Evidence

Screenshots are useful evidence. Record both successful access and denied or hidden functions.

If a Test Fails

Troubleshooting

- Confirm the user is in the intended group.
- Confirm the group menu setting was saved.
- Confirm the user logged out and back in.
- Check whether a menu branch enables several leaf functions together.
- Decide whether the policy or the configuration is wrong.

Logs and Audit Gaps

Section Context: Logs

What we are talking about

Logs help us understand some application behavior, but they may not record every business event. In this lab, the important question is not only "what is logged?" but also "what is missing?"

Hospital lens

After a privacy incident, St. Isidore needs evidence: who logged in, which user context was active, which patient records changed, and whether the access was appropriate. A generic application log may not be enough to answer those questions.

OpenHospital Logging

Configuration

OpenHospital uses `log4j2-spring.properties` for logging configuration. The documented pattern can include user group and user context through `OHUserGroup` and `OHUser` [Informatici Senza Frontiere Onlus, 2026a].

Expected observation

Log lines may include timestamps, severity, errors, startup messages, and sometimes user/group context. They may **not** include every patient, billing, laboratory, or pharmacy create/update.

Find the Log File

Action

Locate the configured log destination. The documented rolling-file appender writes to an `openhospital.log` path controlled by the logging configuration [Informatici Senza Frontiere Onlus, 2026a].

Practical note

If the file is hard to find, enable console output for the lab run, then repeat login and test actions while observing the terminal.

Generate Events

Action

For two different users, repeat one login and one allowed business action. Then attempt one action that should not be available or should fail.

What to collect

Record whether the log shows username, group, timestamp, module, action, warning, or error. If a create/update is **not** logged, record that absence as a finding.

Expected Finding: Log Gaps

DOCTORS WITH AFRICA CUAMM

Important

Do not assume that `openhospital.log` is a complete audit trail. It may be mainly an operational/debug log, not a per-record clinical activity log.

Hospital example

If `lab01` creates a lab result and the log only shows generic application messages, the lab has found an audit gap: access was configured, but record-level accountability is incomplete.

Log Review Questions

Questions

- Can we tell which user performed the action?
- Can we tell which group or role was active?
- Can we identify the affected patient or record?
- Can we distinguish create, update, delete, and read events?
- Can we distinguish allowed work from failed attempts?
- Is the log protected from ordinary users?
- Is this enough for compliance, or only operational debugging?

Audit vs Application Logs

DOCTORS WITH AFRICA CUAMM

Distinction

Application logs are useful, but formal audit usually requires a dedicated audit trail: who did what, to which record, when, from where, and whether the action succeeded.

Hospital example

A real hospital would also centralize logs, restrict access, protect integrity, synchronize time, monitor suspicious access, and define retention and review procedures.

What a Real Audit Trail Should Capture

Minimum useful fields

- **Actor**: user, group, session, and source workstation.
- **Action**: read, create, update, delete, login, logout, permission change.
- **Target**: patient, record, module, or configuration item.
- **Result**: success, denial, validation error, or system error.
- **Time**: synchronized timestamp and retention period.

Checkpoint: Logs

DOCTORS WITH AFRICA CUAMM

You have succeeded when

- You found where OpenHospital writes logs or console output.
- You generated events with at least two users.
- You identified what appears in the log.
- You identified at least one important event that does **not** appear.
- You can explain why logging is not automatically a complete audit trail.

Wrap-Up

Section Context: Wrap-Up

DOCTORS WITH AFRICA CUAMM

What we are talking about

The lab closes by connecting OpenHospital practice back to the theory from lesson-03: RBAC, least privilege, testing, and accountability.

What Model Did We Use?

DOCTORS WITH AFRICA CUAMM

Answer

OpenHospital uses an **RBAC-like** application model. Users are assigned to groups, and groups receive permissions over application menus and functions.

Not DAC, not MAC

It is not primarily DAC because ordinary users are not freely delegating resource ownership. It is not MAC because decisions are not based on formal labels and clearances.

What Administrators Actually Do

Operational lesson

System administrators usually do not implement access-control algorithms from scratch. They choose and configure policies in systems that already provide enforcement mechanisms.

OpenHospital example

The administrator defines groups, assigns users, enables or disables functions, changes default credentials, and checks whether logging is sufficient for the accountability requirement.

Lab Notes

Record

- A list of groups and users created.
- A screenshot or written note for one allowed action per group.
- A screenshot or written note for one denied action per group.
- One log excerpt or console observation.
- One audit-gap finding: an important action that was not clearly logged.
- A short reflection on whether the policy follows least privilege.

Final Recap

Takeaways

- OpenHospital gives a realistic healthcare RBAC-like lab.
- Multi-user mode is required for meaningful accountability.
- Groups should be designed from job responsibilities.
- Permissions must be tested, not merely configured.
- Logs may support review, but missing create/update evidence is itself an audit finding.

References I

[Informatici Senza Frontiere Onlus, 2026a] Informatici Senza Frontiere Onlus (2026a).
Open Hospital 1.15.0 Administrator's Guide.
Informatici Senza Frontiere Onlus
https://www.open-hospital.org/wp-content/OH_manuals/admin/AdminManual.html.

[Informatici Senza Frontiere Onlus, 2026b] Informatici Senza Frontiere Onlus (2026b).
Open Hospital 1.15.0 User's Guide.
Informatici Senza Frontiere Onlus
https://www.open-hospital.org/wp-content/OH_manuals/user/UserManual.html.